OPERATING SYSTEMS – ASSIGNMENT 3 (CO3)

Q1. Linking, Loading and Address Binding

1. Introduction

Linking and loading are important memory-management activities. Linking combines program modules and libraries, while loading places the executable program into memory for execution.

A banking application may contain several modules, such as customer details, account management, and encryption. If the encryption library is updated, dynamic linking can avoid distributing a completely new executable to every branch.

2. Linking

Linking is the process of combining object files and required libraries to create an executable program.

There are two main types.

Static Linking

- **Required library code is copied into the executable.**
- **The executable can run without the original library file being installed separately.**
- **The executable is generally larger.**
- **Updating a library usually requires rebuilding or replacing the executable.**

Dynamic Linking

- **The executable refers to shared libraries.**
- **The shared library is loaded when the program starts or when it is required.**
- **The executable is generally smaller.**
- **Multiple programs can share library code in memory.**
- **Updating a compatible shared library may not require rebuilding the executable.**

3. Static and Dynamic Linking Comparison

Feature	Static linking	Dynamic linking

Executable size	**Larger**	**Smaller**
Library code	**Included in executable**	**Stored in shared library**
Memory usage	**May duplicate library code**	**Code pages can be shared**
Updating	**Rebuild or replace executable**	**Replace compatible library**
Dependency	**Fewer runtime library dependencies**	**Requires shared library**
Startup	**Can avoid dynamic-library loading work**	**May require library loading and relocation**

4. Address Binding

Address binding connects program instructions and data to memory addresses.

Compile-Time Binding

- **Used when the memory location is known during compilation.**
- **The compiler generates address-dependent code.**
- **If the location changes, recompilation may be needed.**

Load-Time Binding

- **The final memory location is decided when the program is loaded.**
- **The loader adjusts addresses before execution.**
- **The program cannot be moved freely during execution without further support.**

Execution-Time Binding

- **Address translation occurs while the program runs.**
- **The process can be moved between memory locations.**
- **Hardware support such as the memory management unit (MMU) is required.**

5. Linker and Loader

Linker	**Loader**
Combines object files	**Loads executable into memory**

Resolves external symbols Creates the process memory image

Connects library references Maps program segments

Produces executable output Starts program execution

6. Position-Independent Code

Position-independent code (PIC) can execute correctly regardless of where it is loaded in memory.

It is commonly used in shared libraries because the same library can be mapped at different virtual addresses in different processes.

7. Static Linking Example

```
#include <stdio.h>


int main()
{
   printf("Banking Application\n");
   return 0;
}
```

8. Dynamic Linking Example

```
gcc bank.c -o bank_dynamic
```

9. Runtime Library Loading

The dlopen() function loads a shared library at runtime. The dlsym() function obtains the address of a function from that library.

Example program:

```
#include <stdio.h>
#include <dlfcn.h>


int main()
{
   void *handle;
   void (*hello)();
```

```
    handle = dlopen("./libhello.so", RTLD_LAZY);

    if (!handle) {
        printf("Library loading failed\n");
        return 1;
    }

    hello = (void (*)())dlsym(handle, "hello");

    if (!hello) {
        printf("Function not found\n");
        dlclose(handle);
        return 1;
    }

    hello();

    dlclose(handle);
    return 0;
}
```

Output:

Hello from shared library

10. Address Observation

The following Linux commands help inspect memory mappings:

cat /proc/self/maps

11. Experimental Investigation

The following experiments can be performed:

1. **Compare the sizes of static and dynamic executables.**
2. **Use ldd to list dynamic dependencies.**

3. **Replace a compatible shared library and test the application again.**
4. **Run two program instances and inspect their memory mappings.**
5. **Rename or temporarily remove a required library in a safe test environment and observe the loading error.**

Actual start-up times and memory usage should be measured on the Linux system. They should not be assumed.

12. Alternative: Runtime Loading with dlopen()

Instead of loading the encryption library when the program starts, the application can call dlopen() only when encryption is required.

Advantages:

- **Reduces library memory use when encryption is not needed.**
- **Allows optional features.**
- **Keeps the initial program loading process simpler when the library is not required.**

Disadvantages:

- **The first encryption request may take longer.**
- **The program must handle missing libraries.**
- **Function lookup errors must be handled.**
- **Library compatibility must be checked.**

13. Final Recommendation

The bank should use dynamic linking for common libraries that are shared by multiple applications. It can use runtime loading for optional encryption features that are not required in every operation.

Q2. Paging

1. Introduction

Paging is a memory-management technique that divides a process's logical memory into fixed-size pages and physical memory into fixed-size frames.

Each page can be placed in any available frame. A page table stores the relationship between logical pages and physical frames.

Paging is useful for a college server that runs many programs of different sizes.

2. External Fragmentation

External fragmentation occurs when free memory is divided into small, separate blocks. A process may not find one sufficiently large contiguous block even when the total free memory is sufficient.

Example:

Physical Memory

+---------+---------+---------+---------+

| Process | Free 2MB| Process | Free 3MB|

+---------+---------+---------+---------+

Total free memory = 5 MB

3. How Paging Works

1. **The CPU generates a logical address.**
2. **The logical address is divided into a page number and an offset.**
3. **The page number is used to access the page table.**
4. **The page table provides the corresponding frame number.**
5. **The frame number and offset form the physical address.**

\[Logical\ Address = Page\ Number + Offset \]

\[Physical\ Address = Frame\ Number \times Page\ Size + Offset \]

4. Address Translation Example

Given:

- **Page size = 1024 bytes**
- **Logical address = 1500**

- **Page 1 is stored in frame 2**

Page number:

$$p = 1500/1024 = 1$$

Offset:

$$d = 1500 \bmod 1024 = 476$$

Physical address:

$$PA = 2 \times 1024 + 476$$

$$PA = 2524$$

5. Page Table Size

For a 32-bit address space and 4 KB pages:

$$4KB=2^{12}\ \text{bytes}$$

Number of pages:

$$\frac{2^{32}}{2^{12}}=2^{20}$$

Therefore, the page table requires 2^{20} entries for a single-level page table covering the full address space.

For a 48-bit address space:

$$\frac{2^{48}}{2^{12}}=2^{36}$$

Therefore, a single-level page table would require 2^{36} entries.

If each entry occupies 4 bytes:

Address space	Number of entries	Page-table size
32-bit	2^{20}	4 MB
48-bit	2^{36}	256 GB

These are theoretical single-level table sizes. Real systems commonly use hierarchical or other page-table designs to avoid allocating such enormous tables.

6. Effective Access Time

A Translation Lookaside Buffer (TLB) is a small, fast cache that stores recent page-table translations.

Assume:

- **Memory access time = 100 ns**

- **TLB lookup time = 10 ns**
- **TLB hit ratio = 95%**

On a TLB hit:

\[10+100=110ns \]

On a TLB miss, assuming one additional memory access for a single-level page table:

\[10+100+100=210ns \]

Effective access time:

\[EAT=0.95(110)+0.05(210) \]

\[EAT=115ns \]

Without a TLB, a single-level page-table lookup generally requires two memory accesses:

\[EAT=100+100=200ns \]

7. Page Size Comparison

Page size	Advantages	Disadvantages
1 KB	Less internal fragmentation per allocation	More page-table entries
4 KB	Balanced size for many workloads	Moderate table overhead
2 MB	Fewer page-table entries and useful for large memory regions	Can waste more memory for small allocations

8. Types of Page Tables

Hierarchical Page Table

- **Divides the page table into multiple levels.**
- **Allocates lower-level tables only when required.**
- **Reduces memory consumption for sparse address spaces.**

Hashed Page Table

- **Uses a hash function to locate page-table entries.**
- **Useful for large address spaces.**
- **Collisions must be handled.**

Inverted Page Table

- **Maintains one entry per physical frame rather than one entry for every virtual page.**
- **Reduces table size.**
- **Translation may require searching or hashing.**

9. Protection and Shared Pages

Paging supports memory protection using page-table permission bits.

Examples:

- **Read permission.**
- **Write permission.**
- **Execute permission.**
- **User and kernel access permissions.**

Shared pages allow multiple processes to map the same physical memory. This is useful for shared libraries and read-only program code.

10. Internal Fragmentation

Internal fragmentation is unused space inside an allocated page.

\[Internal\ Fragmentation = Allocated\ Page\ Space - Required\ Space \]

For example, if a process needs 5,000 bytes and the page size is 4,096 bytes:

Two pages are required.

\[Allocated=8192\ bytes \]

\[Fragmentation=8192-5000=3192\ bytes \]

11. Page Translation C Program

```
#include <stdio.h>


#define PAGE_SIZE 1024

#define PAGES 8


int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};


int main()
```

```
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};

    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];

        unsigned p = a / PAGE_SIZE;
        unsigned d = a % PAGE_SIZE;

        printf("%5u -> page %u offset %6u : ", a, p, d);

        if (p >= PAGES)
            printf("invalid address\n");

        else if (page_table[p] == -1)
            printf("page fault\n");

        else {
            unsigned physical =
                page_table[p] * PAGE_SIZE + d;

            printf("physical %u\n", physical);
        }
    }

    return 0;
}
```

Expected output:

```
    0 -> page 0 offset      0 : physical 5120
```

1500 -> page 1 offset 476 : physical 2524

3100 -> page 3 offset 28 : physical 7196

6200 -> page 6 offset 56 : physical 3128

9000 -> page 8 offset 808 : invalid address

2049 -> page 2 offset 1 : page fault

12. Alternative: Inverted Page Table

An inverted page table maintains one entry for every physical frame.

Advantages:

- **Table size depends on physical memory.**
- **Reduces memory used by page tables.**
- **Suitable for large virtual address spaces.**

Disadvantages:

- **Address translation can require a hash lookup.**
- **Hash collisions increase search work.**
- **Shared memory and process identification require additional handling.**

13. Final Recommendation

For a multi-user college server, 4 KB pages provide a practical balance between memory management and fragmentation. Large pages can be used for selected workloads that benefit from fewer translation entries.

Q3. Segmentation

1. Introduction

Segmentation is a memory-management technique that divides a program into logical parts called segments.

Examples include:

- **Code segment**
- **Data segment**
- **Stack segment**
- **Heap segment**

Each segment has a different purpose and may have a different size.

2. Segment Table

A segment table stores the base address and limit of each segment.

- **Base: Starting physical address of the segment.**
- **Limit: Length or maximum valid offset of the segment.**

3. Address Translation

A logical address contains:

\[(Segment\ Number,\ Offset) \]

The operating system or hardware checks whether the offset is within the segment limit.

If:

\[Offset < Limit \]

then:

\[Physical\ Address = Base + Offset \]

Otherwise, an address-protection error occurs.

4. Example

Segment	Base	Limit
Code	1000	500
Data	3000	800
Stack	5000	400

For Data segment offset 200:

\[PA=3000+200=3200 \]

For Data segment offset 900:

Since 900 is greater than the limit of 800, the address is invalid.

5. Segmentation Program

```
#include <stdio.h>

int main()
{
    int base[] = {1000, 3000, 5000};
    int limit[] = {500, 800, 400};

    int segment, offset;

    printf("Enter segment number: ");
    scanf("%d", &segment);

    printf("Enter offset: ");
    scanf("%d", &offset);

    if (segment < 0 || segment >= 3)
        printf("Invalid segment\n");

    else if (offset < 0 || offset >= limit[segment])
        printf("Segmentation fault\n");

    else
        printf("Physical address = %d\n",
               base[segment] + offset);
```

```
    return 0;
}
```

6. Segmentation with Paging

Segmentation with paging combines the benefits of both techniques.

1. **A program is divided into logical segments.**
2. **Each segment is divided into pages.**
3. **Each segment has a page table.**
4. **The page table maps pages to physical frames.**
5. **The physical address is generated using the frame number and offset.**

This approach supports logical program organisation while avoiding the need for each segment to occupy one continuous physical-memory block.

7. Protection and Sharing

Segmentation supports protection by assigning different permissions to different segments.

For example:

- **Code segment: Read and execute.**
- **Data segment: Read and write.**
- **Stack segment: Read and write.**

Code segments can be shared among processes running the same program, provided that they are mapped with suitable permissions.

8. External Fragmentation

Segmentation uses variable-sized memory blocks. Over time, free memory may become divided into small gaps.

This produces external fragmentation.

Compaction may combine scattered free spaces, but it can be expensive because memory contents may need to be moved.

9. Comparison of Memory Schemes

Feature	Pure paging	Pure segmentation	Segmented paging

Division	**Fixed-size pages**	**Variable-size segments**	**Segments divided into pages**
External fragmentation	**No**	**Yes**	**No at physical page allocation level**
Internal fragmentation	**Possible**	**Usually limited to allocation overhead**	**Possible**
Logical organisation	**Less direct**	**Strong**	**Strong**
Protection	**Page permissions**	**Segment permissions**	**Both levels**
Sharing	**Page-based**	**Segment-based**	**Segment and page-based**

10. Linux Memory Mapping

Linux represents a process's memory regions through /proc.

Use:

cat /proc/PID/maps

The output may show:

- **Executable code**
- **Shared libraries**
- **Heap**
- **Stack**
- **Memory-mapped files**

Modern Linux primarily uses paging for virtual-memory management. The memory-map regions reflect logical areas of a process, rather than a traditional pure segmentation scheme.

11. Segmentation Fault

A segmentation fault occurs when a process accesses memory in an invalid or unauthorised way.

For example, accessing memory outside an allocated array or writing to read-only memory can cause a segmentation fault.

The operating system detects the invalid access and normally sends the process a SIGSEGV signal.

12. Alternative: Pure Paging

If the system used pure paging instead of segmentation:

- **All memory would be divided into fixed-size pages.**
- **Logical addresses would be translated using page tables.**
- **External fragmentation would be avoided.**
- **Internal fragmentation could remain.**
- **Protection and sharing would be implemented through page-table permissions.**

Q4. Virtual Memory and Demand Paging

1. Introduction

Virtual memory allows a process to use an address space larger than the available physical RAM.

Demand paging loads a page into RAM only when the process actually accesses it.

A medical-imaging program requiring 24 GB of memory can run on a computer with 16 GB of RAM if its active working set fits in memory and sufficient backing storage is available.

2. Demand Paging

In demand paging, pages are loaded into physical memory when needed.

Advantages:

- **Reduces initial memory loading.**
- **Allows large programs to run with limited RAM.**
- **Avoids loading unused pages.**
- **Improves memory utilisation.**

3. Page Fault Handling

A page fault occurs when a process accesses a page that is not currently available in physical memory.

Steps:

1. **The CPU generates a memory address.**
2. **The MMU detects that the page is not present.**
3. **A page-fault exception occurs.**
4. **The operating system checks whether the memory access is valid.**
5. **The OS finds a free frame or selects a page for replacement.**
6. **The required page is loaded from storage.**
7. **The page table is updated.**
8. **The interrupted instruction is restarted.**

4. Effective Access Time

Given:

- **Memory access time = 100 ns**

- **Page-fault service time = 8 ms**

Convert:

$$ 8ms=8,000,000ns $$

Formula:

$$ EAT=(1-p)\times Memory\ Time+ p\times Page\ Fault\ Time $$

where p is the page-fault rate.

Case 1: One fault in 1,000 accesses

$$ p=0.001 $$

$$ EAT=0.999(100)+0.001(8000000) $$

$$ EAT=8099.9ns $$

Case 2: One fault in 100,000 accesses

$$ p=0.00001 $$

$$ EAT=0.99999(100)+0.00001(8000000) $$

$$ EAT=179.999ns $$

Case 3: One fault in 1,000,000 accesses

$$ p=0.000001 $$

$$ EAT=0.999999(100)+0.000001(8000000) $$

$$ EAT=107.9999ns $$

5. Effective Access Time Table

Page-fault rate	Effective access time
1 in 1,000	8,099.9 ns
1 in 100,000	179.999 ns
1 in 1,000,000	107.9999 ns

The results show that even a small page-fault rate can significantly increase memory-access time because storage access is much slower than RAM access.

6. Copy-on-Write

Copy-on-Write (COW) is a technique used when a process is duplicated.

Instead of immediately copying every memory page, the parent and child initially share the same physical pages.

When one process attempts to modify a shared page:

1. **A protection fault occurs.**
2. **The OS creates a private copy of the page.**
3. **The process modifies its own copy.**

This reduces the cost of fork() when the child does not modify most of the parent's memory.

7. Thrashing

Thrashing occurs when the system spends too much time moving pages between RAM and storage rather than executing processes.

Causes:

- **Insufficient physical memory.**
- **Too many active processes.**
- **Poor locality of reference.**
- **Insufficient frames allocated to processes.**

Effects:

- **High page-fault rate.**
- **Increased disk activity.**
- **Low CPU utilisation.**
- **Slow application response.**

8. Working-Set Model

The working set is the group of pages actively used by a process during a recent time window.

The operating system can use working-set information to estimate how many frames a process requires.

If the total working-set demand exceeds available physical frames, the OS may reduce the number of active processes or adjust memory allocation.

9. Frame Allocation

Equal Allocation

Each process receives the same number of frames.

Proportional Allocation

Processes receive frames based on their memory requirements or size.

Method	Advantage	Disadvantage
Equal allocation	Simple	Large processes may need more frames
Proportional allocation	Better matches memory needs	Requires tracking process sizes

10. Experimental Investigation

The following experiments can be performed:

getconf PAGESIZE

11. Alternative: 32 GB RAM

If the laptop has 32 GB RAM, the 24 GB application can potentially keep most of its working data in physical memory.

Compared with a 16 GB machine:

- **Fewer page faults may occur.**
- **Storage access may be reduced.**
- **Application response may improve.**
- **The system has more room for other processes.**

However, the benefit depends on the actual working set and memory usage.

12. Final Recommendation

The radiologist should first improve the program's memory locality and remove unnecessary memory usage. If the application consistently requires a large working set, increasing RAM to 32 GB is a practical solution.

Q5. Page-Replacement Techniques

1. Introduction

Page replacement is used when a process requires a page but no free physical frame is available.

The operating system selects an existing page to remove and loads the required page.

A web server frequently accesses stored data pages. A suitable page-replacement algorithm helps reduce page faults and improve performance.

2. FIFO Page Replacement

FIFO stands for First In First Out.

- **The oldest loaded page is replaced first.**
- **It is simple to implement.**
- **It does not consider how frequently or recently a page is used.**
- **It may suffer from Belady's anomaly.**

3. LRU Page Replacement

LRU stands for Least Recently Used.

- **Replaces the page that has not been used for the longest time.**
- **Uses recent access history.**
- **Performs well when recent usage predicts future usage.**
- **Does not suffer from Belady's anomaly.**

4. Optimal Page Replacement

Optimal replacement removes the page whose next use is farthest in the future.

- **Produces the minimum possible page faults for a known reference string.**
- **Requires knowledge of future page references.**
- **Is mainly used as a theoretical comparison.**

5. Reference String

The assignment provides:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3,

0, 3, 2, 1, 2, 0, 1, 7, 0, 1

6. Page Fault Results

Frames	FIFO	LRU	Optimal
3	15	12	9
4	10	8	8

The results show that LRU performs better than FIFO for this reference string. Optimal produces the minimum number of page faults.

7. LRU C Program

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++)
        if (f[i] == page)
            return i;

    return -1;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF];
    int used = 0, faults = 0;

    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);

        if (k >= 0) {
```

```
            last[k] = i;
            continue;
        }

        faults++;

        if (used < frames) {
            f[used] = ref[i];
            last[used] = i;
            used++;
        } else {
            int victim = 0;

            for (int j = 1; j < frames; j++)
                if (last[j] < last[victim])
                    victim = j;

            f[victim] = ref[i];
            last[victim] = i;
        }
    }

    return faults;
}

int main()
{
    int ref[] = {
        7, 0, 1, 2, 0, 3, 0, 4, 2, 3,
```

```
    0, 3, 2, 1, 2, 0, 1, 7, 0, 1
};

int len = 20;

printf("LRU 3 frames = %d\n", lru(ref, len, 3));
printf("LRU 4 frames = %d\n", lru(ref, len, 4));

return 0;
}
```

8. Belady’s Anomaly

Belady's anomaly occurs when increasing the number of frames increases the number of page faults.

The assignment gives the reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO results:

Frames	Page faults
3	9
4	10

The number of faults increases from 9 to 10 even though the number of frames increases.

This is Belady's anomaly.

9. Why LRU Does Not Have Belady’s Anomaly

LRU is a stack algorithm.

The pages held with \(n\) frames form a subset of the pages held with \(n+1\) frames for the same reference history.

Therefore, increasing the number of frames cannot increase the number of page faults under LRU.

10. Clock Algorithm

The Clock algorithm is also called Second-Chance replacement.

Each page has a reference bit.

Working:

1. **The algorithm checks the page pointed to by the clock hand.**
2. **If its reference bit is zero, the page can be replaced.**
3. **If its reference bit is one, the bit is cleared and the page receives a second chance.**
4. **The clock hand moves to the next page.**
5. **The process repeats until a replacement candidate is found.**

11. Comparison of Algorithms

Feature	FIFO	LRU	Optimal	Clock
Replacement basis	Oldest loaded	Least recently used	Farthest future use	Reference bit
Implementation	Simple	More complex	Not practical for future knowledge	Moderate
Belady's anomaly	Possible	No	No	Not guaranteed to be absent
Performance	Depends on order	Good for locality	Best theoretical	Practical approximation

12. Complexity Analysis

For a simple implementation using a linear search through frames:

- **Searching for a page takes \(O(f)\), where \(f\) is the number of frames.**
- **LRU victim selection takes \(O(f)\).**
- **Processing \(n\) references takes \(O(nf)\).**

More advanced implementations can use hash tables and linked lists to improve lookup and replacement efficiency.

13. Experimental Investigation

The assignment asks for:

- **Strong-locality reference strings.**
- **Random reference strings.**
- **Frame counts from 3 to 10.**

- **Running-time measurements for 100,000 references.**

A measurement table can be maintained as follows:

Frames	FIFO faults	LRU faults	Optimal faults
3	Record result	Record result	Record result
4	Record result	Record result	Record result
5	Record result	Record result	Record result
6	Record result	Record result	Record result
7	Record result	Record result	Record result
8	Record result	Record result	Record result
9	Record result	Record result	Record result
10	Record result	Record result	Record result

Actual experimental values must be collected by executing the algorithms.

14. Alternative: Clock Instead of Exact LRU

The Clock algorithm uses a reference bit rather than maintaining exact access timestamps or a complete recency list.

Advantages:

- **Lower bookkeeping overhead.**
- **Requires approximately one reference bit per frame.**
- **Suitable for practical operating-system implementations.**

Disadvantages:

- **It only approximates LRU.**
- **It may replace a page that was used more recently than another page.**
- **Its page-fault count can be higher than exact LRU.**

15. Final Recommendation

For a web server, LRU or a Clock-based approximation is suitable. LRU performs well for workloads with strong locality, while Clock reduces the overhead of maintaining exact usage order.

Q6. Disk Management

1. Introduction

Disk management is the process of organising, allocating, and maintaining secondary storage devices.

A hospital records server requires reliable storage because patient records must remain available even if a disk fails.

2. Low-Level Formatting

Low-level formatting prepares a disk into sectors that can store data.

It organises the storage device into addressable units used by the disk controller.

3. Partitioning

Partitioning divides a physical disk into separate logical regions.

For example:

- **System partition**
- **Data partition**
- **Backup partition**
- **Swap partition**

4. Logical Formatting

Logical formatting creates a file system on a partition.

Examples include:

- **ext4**
- **XFS**
- **NTFS**
- **FAT32**

The file system organises files, directories, metadata, and free space.

5. Boot Block and Boot Process

The boot block contains information or code required to begin the boot process.

Boot process:

1. **The computer starts.**

2. **BIOS or UEFI performs initial hardware checks.**
3. **The firmware selects a boot device.**
4. **A bootloader is started.**
5. **The bootloader loads the operating-system kernel.**
6. **The kernel initialises memory, devices, and system services.**
7. **The operating system becomes ready for use.**

6. Bad Block Handling

A bad block is a storage sector that cannot reliably store or retrieve data.

The operating system and storage device may handle bad blocks using:

- **Bad-block detection.**
- **Sector remapping.**
- **Spare sectors.**
- **File-system checks.**
- **Backup and recovery.**

Regular monitoring helps identify storage problems before they cause serious data loss.

7. Swap Space

Swap space is storage used as an extension of RAM.

It may be implemented as:

- **A dedicated swap partition.**
- **A swap file.**

A swap partition provides a dedicated area, while a swap file is easier to create, resize, and manage.

For a hospital server, sufficient physical RAM and reliable storage should be prioritised. Swap should not be treated as a replacement for adequate RAM.

8. RAID Levels

RAID stands for Redundant Array of Independent Disks.

RAID level	Method	Minimum disks	Fault tolerance
RAID 0	Striping	2	No disk-failure protection

RAID 1	**Mirroring**	**2**	**Can survive a disk failure in a two-disk mirror**
RAID 5	**Striping with distributed parity**	**3**	**Can survive one disk failure**
RAID 10	**Mirroring and striping**	**4**	**Can survive multiple failures if they do not destroy a mirror pair**

9. Capacity of Six 4 TB Disks

Total raw capacity:

\[6 \times 4 = 24TB \]

RAID level	**Usable capacity**
RAID 0	**24 TB**
RAID 1	**12 TB for three mirrored pairs**
RAID 5	**20 TB**
RAID 10	**12 TB**

These values assume equal-sized disks and decimal TB units. RAID 1 capacity depends on the mirror layout; three mirrored pairs provide 12 TB.

10. RAID Comparison

RAID 0

- **High performance.**
- **Uses all disk capacity.**
- **No redundancy.**
- **Failure of one disk can destroy the array.**

RAID 1

- **Stores duplicate data.**
- **Provides strong redundancy.**
- **Uses approximately half the raw capacity in mirrored pairs.**

RAID 5

- **Uses distributed parity.**
- **Provides good capacity efficiency.**

- **Can survive one disk failure.**
- **Rebuilding after failure can place significant stress on remaining disks.**

RAID 10

- **Combines striping and mirroring.**
- **Provides good read and write performance.**
- **Offers redundancy.**
- **Uses half the raw capacity.**

11. Swap Partition vs Swap File

Feature	Swap partition	Swap file
Location	Dedicated partition	File within a file system
Setup	Requires partitioning	Easier to create
Resizing	More difficult	Usually easier
Management	Separate storage area	Managed like a file
Flexibility	Lower	Higher

12. Linux Disk Commands

Display block devices:

Lsblk

Display memory and swap:

free -h

13. Alternative: RAID 6

RAID 6 uses two independent parity blocks.

For six disks of 4 TB:

\[Usable\ Capacity=(6-2)\times4 \]

\[=16TB \]

Advantages:

- **Can survive two disk failures.**
- **Provides stronger fault tolerance than RAID 5.**
- **Suitable for large storage systems where rebuilding takes time.**

Disadvantages:

- **Less usable capacity.**
- **More parity calculations.**
- **Write performance can be lower than RAID 5 or RAID 10.**

14. Final Recommendation

For a hospital records server, RAID 10 or RAID 6 is suitable depending on performance and capacity requirements.

- **RAID 10 is appropriate when high performance and fast recovery are important.**
- **RAID 6 is appropriate when surviving two disk failures is a priority.**

RAID must be combined with independent backups because RAID alone does not protect against accidental deletion, malware, or site-level disasters.

Q7. Design a Memory-Management Plan from First Principles

1. Introduction

Memory management is an important operating-system function that allocates, protects, and monitors memory used by processes.

A smart factory edge device has 2 GB of RAM and no swap disk. It runs three tasks:

1. **Real-time control task.**
2. **Machine-learning task requiring 1.5 GB.**
3. **Logging task.**

The real-time task must not be delayed by page faults. Therefore, the memory-management plan must prioritise predictable execution.

2. Stage 1 – Requirements Analysis

Task	Memory requirement	Timing requirement	Page-fault tolerance
Real-time control	Small and predictable	Strict deadline	Cannot tolerate page faults during critical execution
Machine-learning	Approximately 1.5 GB	High memory demand	May tolerate occasional faults
Logging	Depends on log volume	Lower priority	Can tolerate some delay

3. Measurable Requirements

The design should satisfy these requirements:

- **The real-time task must have its required memory allocated before critical execution.**
- **Its critical memory pages must remain resident.**
- **The machine-learning task must stay within its assigned memory budget.**
- **Logging must not consume memory required by the control task.**
- **The system must prevent uncontrolled memory allocation.**
- **New tasks must be admitted only after checking available memory.**

4. Stage 2 – Compare Memory-Management Mechanisms

Contiguous Allocation

- **Allocates one continuous block to a process.**
- **Simple to understand.**
- **Can suffer from external fragmentation.**
- **Resizing a block may be difficult.**

Paging

- **Divides memory into fixed-size pages.**
- **Avoids external fragmentation.**
- **Supports memory protection.**
- **Page faults may cause unpredictable delays.**

Segmentation

- **Divides a program into logical regions.**
- **Supports different protection rules for code, stack, and data.**
- **Can suffer from external fragmentation.**

Demand Paging

- **Loads pages only when accessed.**
- **Reduces initial memory requirements.**
- **May cause page-fault delays.**
- **Not suitable for a time-critical loop that cannot tolerate page faults.**

5. Comparison Table

Mechanism	Real-time control	Machine-learning	Logging
Contiguous allocation	Predictable if allocated in advance	May be difficult for large memory	Suitable for small fixed buffers
Paging	Suitable with locked pages	Suitable	Suitable
Segmentation	Supports logical protection	Less flexible for large dynamic memory	Suitable

Demand paging	Avoid for critical execution	Useful for non-critical data	Acceptable when delay is allowed

6. Memory Allocation Plan

The 2 GB RAM should be divided using a reserved memory budget.

An illustrative plan is:

Component	Memory budget
Real-time control task	128 MB
Machine-learning task	1,500 MB
Logging task	64 MB
Operating system and drivers	256 MB
Remaining reserve	100 MB
Total	2,048 MB

This is an example allocation plan. The actual allocation must be validated against the real task requirements and operating-system overhead.

7. Memory Locking with mlock()

The mlock() system call locks a memory region into RAM so that the pages are not swapped out.

For a time-critical task, the program can allocate its memory in advance and lock the required pages.

C program:

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/mman.h>


#define SIZE 1024 * 1024


int main()

{

   char *memory = malloc(SIZE);
```

```
    if (!memory) {
        perror("malloc");
        return 1;
    }

    if (mlock(memory, SIZE) != 0) {
        perror("mlock");
        free(memory);
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        memory[i] = 0;

    printf("Memory locked successfully\n");

    munlock(memory, SIZE);
    free(memory);

    return 0;
}
```

Compilation:

```
gcc memory_lock.c -o memory_lock
```

8. Why Memory Locking Is Important

- **Prevents locked pages from being swapped out.**
- **Reduces page-fault-related delays during critical execution.**
- **Makes memory behaviour more predictable.**
- **Helps time-critical applications meet deadlines.**

However, mlock() alone does not guarantee real-time execution. CPU scheduling, interrupts, driver delays, and other kernel activities can still affect execution time.

9. Memory Simulation

The system can simulate the memory usage of all three tasks.

Task	Example memory usage
Real-time control	128 MB
Machine-learning	1,500 MB
Logging	64 MB
Total task usage	1,692 MB

Remaining memory:

\[2048-1692=356MB \]

The remaining memory must also support the operating system, drivers, and other required services. The allocation plan should be adjusted if actual usage exceeds the budget.

10. Verification

The following tests can be performed:

1. **Run the control task without memory locking.**
2. **Record execution-time measurements.**
3. **Run the control task with memory locking.**
4. **Record execution-time measurements again.**
5. **Compare average and worst observed delays.**
6. **Repeat the test under machine-learning and logging workloads.**

Test	Memory locking	Observed worst-case delay
Test 1	Disabled	Record measured result
Test 2	Enabled	Record measured result

The values must come from actual execution. A single observed maximum is not proof of a guaranteed worst-case execution time.

11. Stage 4 – Fourth Task of 500 MB

If a fourth task requiring 500 MB is added:

- **Total task memory demand increases.**
- **The system may not have enough free memory.**
- **The machine-learning task may need a smaller memory budget.**
- **The new task should not be allowed to interfere with the real-time task.**
- **Memory admission control must be performed before starting the new task.**

The system may need to move the machine-learning task to a separate device or reduce its memory usage.

12. Alternative: Real-Time Operating System

A real-time operating system without virtual memory can provide more predictable memory behaviour.

Feature	Linux with locked memory	RTOS without virtual memory
Memory management	Virtual memory with selected locked pages	Usually fixed physical-memory allocation
Page-fault delay	Avoided for locked pages after preparation	No demand-paging faults
Real-time predictability	Depends on configuration and workload	Often designed for predictable timing
Machine-learning task	Can use remaining memory	Limited by fixed RAM
Adding new tasks	More flexible	Requires careful memory planning

13. Final Recommendation

The edge device should reserve memory for the real-time control task, lock its critical pages using mlock(), and prevent lower-priority tasks from consuming its memory budget.

The machine-learning task should use controlled memory allocation, while logging should use a limited buffer.

For applications with extremely strict timing requirements, a real-time operating system may be more suitable than a general-purpose Linux configuration.